

Interactive Systems – Final Project

Final Report

February 21, 2026

Submitted to: Prof. Hartmut Seichter, Ph.D.

Preamble

Authors

Name	Student ID	GitHub
Sarthak D. Bharad	318117	SarthakBharad 

Declaration

I hereby declare that:

1. This report is my own genuine work.
2. I have not submitted materials from this project to other courses. I have not collaborated on this work with students from other courses outside of Computer Graphics (Prof. Seichter).
3. I have adhered to the requirements in regard of AI usage. My use of Artificial Intelligence tools is disclosed as follows:

*A combination of Claude and ChatGPT was used to assist with **commenting** the scripts (the code itself is **NOT** AI generated), the **README** file on GitHub and to create this template for better readability and visual appeal.*

Date: 21.02.2026 **Signature:** Sarthak D. Bharad

Contents

1	Goal Description	3
2	Theoretical Foundations	4
3	Inspirations	6
4	Development Process	8
4.1	Steps Taken and Milestones	8
4.2	Variants and Solutions Explored	10
4.3	Conclusion and Lessons Learned	12
A	Literature and References	14
B	Resources and Assets	15
B.1	Audio Assets	15
B.2	Visual Assets and Textures	16
B.3	Code Libraries and Frameworks	16
B.4	Tools and Utilities	16
B.5	Online Resources	17
C	Project Repository	18
C.1	Repository Structure	18
C.2	Win Condition Constants	19
C.3	How to Run	19

1 Goal Description

The primary objective of this project is to develop a fully functioning 3D platformer game using the Godot game engine. I have named it **Gilded Loop**, as there is only a single level as of now due to time constraints. The game challenges players to navigate a three-dimensional world built from elevated platforms, collecting coins and defeating enemies scattered throughout to complete the level.

Gameplay Mechanics: Several gameplay mechanics have been implemented to enhance the player's nostalgic experience and they are as follows:

Action	Input
Movement	WASD or Arrow Keys
Jump	Spacebar or Enter
Camera Rotation	Numpad 1 / Numpad 2
Screenshot	B

Target Audience: The game aims to appeal to the nostalgia of casual to intermediate gamers who enjoy classic, arcade-style 3D platformers; Specially designed for players who appreciate precise movement, spatial awareness challenges.

Key Features and Functionality:

- Movement, Jumping, and Gravity simulation in a 3D world.
- Camera with manual rotation and a smooth Third-Person perspective.
- Jump-based enemy kill alongside a coin collection system with audio-visual feedback and HUD integration.
- Strict win conditions for encouraging skill-based gameplay.
- Moving platforms with configurable motion parameters.
- Floor-alignment that tilts the character model to match surface normals.
- Audio system handling background music, SFX, and various scene-specific audio states.

Success Criteria:

- Responsive character movement, jumping, and camera control.
- Reliable collision detection that differentiates stomp kills from contact deaths.
- Clear visual and audio feedback for all player interactions.
- Level design that strikes a balance between strict and achievable win conditions.
- Stable scene transitions between menu, gameplay, game-over, and win states.

2 Theoretical Foundations

Game Design Principles: Gilded Loop is built upon the foundations of the platformer genre as established by classic titles such as **Super Mario Bros.** and **Crash-Bandicoot**. Key design principles include:

- **Responsiveness:** Accurate jump mechanics and low-latency controls are essential to fair platforming. Players must trust their inputs to make consistent, repeatable decisions about platform traversal and enemy interaction.
- **Feedback Loops:** Every significant player action – collecting a coin, stomping an enemy, falling off a platform – is paired with both audio and visual feedback. This reinforces player actions and communicates game state clearly without relying on text alone.
- **Clear Win Conditions:** A dual objective (in our case, collecting all coins AND defeating all enemies) encourages thorough exploration and prevents players from ignoring either of the gameplay systems.
- **Do or Die:** The stomp mechanic rewards skilled aerial attacks with a score increment and a bounce, while side contact results in immediate failure. This asymmetry creates engaging risk-reward decision-making around enemy encounters.

Godot Engine Architecture: The game leverages Godot 4's Scene/Script system across the following node types:

- **CharacterBody3D:** Used for both the player and enemy movement.
- **AnimatableBody3D:** Used for moving platforms. Specifically designed for objects that should push and carry other physics bodies, making it ideal for moving platforms without requiring RigidBody physics.
- **Area3D:** Used for coins and for the enemy's collision zones.
- **AnimationPlayer:** Drives various animations throughout the project.
- **Tweens:** Used for smooth animation created via scripts.
- **Autoloads:** Provides globally accessible scripts that are accessible in all scenes.

Programming Patterns Used:

- **Singleton:** The `Global` and `SoundManager` scripts act as global services, giving all scripts access to a shared state and audio without having to hard-code dependencies.

- **Component-based Design:** Each game entity is encapsulated in its own scene and script rather than embedding all logic in one single scene or script allowing for reusability and maintainability.
- **Signals:** Godot signals are being used throughout to inform neighbouring scenes of state changes. They notify of state changes the moment they occur.
- **Sequencing:** GDScript's `await` keyword is used in a couple of instances to create time-based sequences without blocking the main thread.

Key Algorithms and Systems:

- **Gravity Integration:** Gravity has been applied to each physics object i.e. when the character is jumping, they are brought back down using `velocity += get_gravity() * delta`, that reads the gravity value from project physics settings.
- **Camera-Relative Movement:** The player's movement direction is computed by projecting the 2D input vector through the camera controller's `transform.basis`, ensuring the character always moves relative to the current camera angle.
- **Floor Alignment:** The character's orientation is continuously aligned to the surface normal using cross products and basis orthonormalization, enabling the character to tilt naturally on slopes via `global_transform.interpolate_with()`.
- **Edge Detection:** A downward-facing `RayCast3D` detects when the enemy is approaching a platform edge. If the raycast stops colliding while the enemy is grounded, the `turn_around()` function is called and the enemy walks off.
- **Platform Oscillation:** Moving platforms tween between two exported `Vector3` positions using `TRANS_CUBIC` easing, producing a predictable motion that players can time their jumps around.

3 Inspirations

Classic Game Inspirations

The project draws heavily from the golden age of console platformers, both mechanically and philosophically:

- **Super Mario Bros. (1985):** The foundational reference for the genre. The coin collection mechanic, the enemy stomp system, and the clear binary win/lose state all trace their roots to this title. The enemy's dual collision zone design (top vs. side contact) is a direct reimplement of the Mario enemy interaction model.
- **Super Mario 64 (1996):** The primary inspiration for the 3D execution. The third-person camera with manual rotation, the exploration-driven level layout, and the combination of collection and combat objectives to unlock progression all reflect Super Mario 64's design philosophy. The camera rotation using fixed 30-degree increments echoes early fixed-camera 3D platformers before analogue stick cameras became standard.
- **Crash Bandicoot Series (1996–):** The emphasis on predictable enemy movement patterns, moving platform timing puzzles, and a single enclosed level with all collectibles visible to the player reflects Crash Bandicoot's design sensibility. The use of asset packs with consistent aesthetic style also mirrors the franchise's focus on visual coherence.

Other Influences

Contemporary independent 3D platformers shaped several design decisions:

- **Responsiveness over Simulation:** Rather than implementing physically accurate inertia, the character uses `move_toward()` for instant acceleration on input release. This prioritises responsiveness and control over realism, valuing player agency.
- **Simple Difficulty Curve:** Enemy AI is kept deliberately simple and predictable. Players can learn enemy patterns in a few observations, which is in line with platformers that favour learnable challenge over random or complex AI behaviour.

Design Inspiration

The visual direction combines multiple curated sources to form a cartoonish aesthetic, with the focus to convey that none of this is serious, and if a player has failed, they can always try again and continue to have fun:

- The robot player character (*Robot Goes to Jersey City* by InfameDavid on BlendSwap) brings a mechanical, stylised protagonist meant to serve as a lighthearted, non-threatening character that fits the game's overall tone.
- The Quaternius Platformer Game Kit provides the enemy models, which feature a consistent low-poly stylised aesthetic, meant to be more of a cute pokemon rather than a menacing threat.
- Custom world geometry with AmbientCG tile textures gives the environment a clean, colourful appearance that clearly distinguishes walkable surfaces from the background.

4 Development Process

4.1 Steps Taken and Milestones

The development followed an iterative, milestone-driven approach, building from a minimal playable prototype towards a polished, feature-complete game.

Phase 1: Project Setup and Planning

The project was initialised as a Godot 4.4.1 project with the standard folder structure immediately established:

`scenes/`, `scripts/`, `assets/`, and `resources/`.

Core design decisions were taken much early; such as the player and enemies being `CharacterBody3D`, enemies having two collision bodies for top vs. side detection, win conditions being a global state, and much more. Godot 4.4.1 (stable) was selected for its mature 3D physics pipeline and the improved `CharacterBody3D` API compared to Godot 3's `KinematicBody`. Another reason to select this engine version was that it was the latest available version of the engine at the time of development.

Phase 2: Character

The character controller was the first and most fundamental system built. Key implementation steps included:

- Input mapping left, right, backward, forward, jump, camera rotate left, camera rotate right, and screenshot to the appropriate keys.
- Camera-relative movement to project the 2D input direction into 3D world space.
- Jump velocity and gravity integration.
- `AnimationPlayer` integration for idle, run, and jump animations.
- Floor normal alignment using cross-product basis construction and `interpolate_with()` for smooth slope transitions.

Phase 3: Enemy AI and Coin System

While the character controller was undergoing development, I also started working on the other two core systems:

The coin system extends `Area3D` and increments `Global.coins` on body entry, plays the collection sound, updates the HUD label, and checks the win condition (such is also the case for the enemy as well). A `bounce` animation plays before the coin is freed with

`queue_free()`, providing satisfying visual feedback. Coins also rotate continuously with a speed of 2 degrees per frame.

The enemy AI uses a `CharacterBody3D` with a downward `RayCast3D` for edge detection and wall detection via the function `is_on_wall()`. When either condition triggers, `turn_around()` initiates a tween-driven 180-degree rotation and reverses the direction vector. Two child `Area3D` nodes handle the interaction logic with the character: landing on top plays a death animation and calls `body.bounce()` on the player; side contact triggers an immediate game over.

Phase 4: Platform System and Level Design

Moving platforms were implemented using scripts by implementing `AnimatableBody3D` and Godot's Tween system with `TRANS_CUBIC` easing. Each platform exposes four `@export` variables (`initial_position`, `final_position`, `time`, `pause`) configurable per instance in the editor, enabling a wide variety of platform behaviours from a single reusable scene.

The level that is present in the game as of now (`level_one.tscn`) was designed to make the most of everything that had been implemented up until now. With 67-coins to collect, 14-enemies to defeat with platforms at varying heights, the level was designed to encourage exploration and create natural spatial challenges.

Phase 5: UI, Audio, and Polish

The HUD displays live coin and enemy counters using panels, updated directly from their respective scripts on each interaction.

The `SoundManager` centralises all audio with dedicated functions for each sound event: `play_jump_sound()`, `play_coin_sound()`, `play_squash_sound()`, `play_game_music()`, and several others.

Scene transitions between menu, gameplay, game over, and win screens are handled via `get_tree().change_scene_to_file()`. A screenshot utility in `global.gd` captures the viewport with `RenderingServer.frame_post_draw` and saves a timestamped PNG (the path has been hard-coded for now, so every player will have to set their own path before playing the game for the first time, but i aspire to make it dynamic in the future).

4.2 Variants and Solutions Explored

Technical Variants

Feature	Option A	Option B	Chosen Alternative
Player Physics	RigidBody3D with force-based movement	CharacterBody3D with manual velocity	CharacterBody3D
Enemy Movement	NavigationAgent3D pathfinding	Directional patrol with wall/edge detection	Directional patrol
Platform Motion	_physics_process with move_and_collide	AnimatableBody3D with Tween	Tween-based
Camera Control	Mouse-look free camera	Fixed-increment manual rotation	Fixed 30° snaps
Gravity	Manual <code>velocity.y -= gravity * delta</code>	<code>get_gravity() * delta</code>	<code>get_gravity()</code>
Global State	Scene-scoped variables passed by reference	Autoload Singleton (Global)	Autoload Singleton

Table 1: Technical variants explored during development

Design Iterations

Jump Feel: Multiple values for `JUMP_VELOCITY` and gravity multiplier were tested during development. The final values (`JUMP_VELOCITY` = 11, default Godot gravity at approximately 9.8 units/s²) were chosen after iterative playtesting to produce a jump arc that is readable and feels snappy without being floaty.

Camera Rotation Increment: Smaller increments (15°) were tested but felt too granular and made orientation confusing. Larger increments (45°, 90°) were too coarse for fine navigation. The 30-degree snap was selected as the best balance between granularity and usability.

Win Condition Design: An early design required only coins for the win condition. Enemy defeats were added as a second requirement to prevent players from simply running past enemies, encouraging engagement with the stomp mechanic and making the full level exploration necessary.

Failed Experiments

Mouse-Look Camera: A free mouse-look camera was prototyped early in development. While more conventionally expected in 3D games, it introduced complexity in determining the correct movement direction relative to the camera, and made the stomp mechanic significantly harder to aim. The fixed-increment rotation system proved more predictable for a precise platformer context.

Double Jump: A double-jump mechanic was prototyped by tracking a boolean. While it made some platform gaps more accessible, it reduced the required precision of jump timing, undermining the core challenge of the game. It was removed in favour of precise single-jump platforming.

Enemy Pathfinding: NavigationAgent3D-based pathfinding was explored for enemies. The result was unpredictable enemy movement that made both the stomp mechanic and the side-contact hazard difficult to anticipate. Simple directional patrol with wall and edge reversal produced fairer, more readable enemy behaviour.

Telemetry and Analytics (Work in Progress)

An additional objective of this project was the integration of a telemetry and analytics subsystem to record player behavior, performance metrics, and gameplay events (e.g., level completion time, deaths, input latency, and interaction frequency). The intention was to automatically log these events to structured JSON files for subsequent analysis and visualization using external tools like python libraries.

Due to time constraints and the prioritization of core gameplay stability and feature completeness, the telemetry system was not fully finalized for inclusion in the submitted build. However, the system design and implementation have been actively developed in parallel. A dedicated analytics manager, session-based logging format, and an external Python analysis pipeline have already been prototyped. These components enable per-session data collection and the generation of visual reports such as event distributions, performance trends, and input-to-render latency measurements.

The telemetry framework is therefore considered partially implemented and serves as a foundation for future iterations of the project, where it will support data-driven balancing, usability evaluation, and performance optimization. The files themselves however, have not been included in the final submission.

Challenges and Solutions

Jump Responsiveness: Achieved through calibration of gravity and velocity rather than physically accurate inertia.

Camera-Relative Movement: Solved by projecting input vectors through the camera's transform basis.

Slope Alignment: Character orientation reconstructed each frame using floor normals and orthonormalized bases.

Enemy Interaction: Two separate `Area3D` nodes (`TopChecker` and `SideChecker`) cleanly differentiated stomp kills from damage.

3D Asset Integration: External models were validated and corrected in Blender before import to ensure scale, orientation, and animation compatibility.

Moving Platform Carry Behavior: Switching from `StaticBody3D` to `AnimatableBody3D` allowed the physics engine to correctly transfer motion to the player.

Screenshot Paths: Replaced hard-coded paths with `OS.get_user_data_dir()` for portability across systems.

4.3 Conclusion and Lessons Learned

Project Outcomes

The final project delivers:

- A responsive 3D player controller with camera-relative movement
- A third-person camera with discrete rotation
- Enemy AI with wall and edge detection
- Coin collection with audio-visual feedback
- Configurable moving platforms
- A global state singleton for win tracking
- A centralized audio manager
- Complete scene flow (menu, gameplay, game over, win)
- Timestamped screenshot capture utility
- Clean modular architecture using focused scripts

Some planned features remain future work, including additional levels, save/load systems, visual effects polish, expanded animations, and standalone builds.

Lessons Learned

This project strengthened both technical and organizational skills:

- Practical experience with Godot 4.x and 3D physics
- Tween-based animation workflows
- Signal-driven system communication
- Autoload singleton design patterns
- Asset import pipelines
- Coroutine-style sequencing using `await`

From a project management perspective, early prototyping of the player controller, modular script design, and minimizing setup friction proved especially valuable.

Future Improvements

Future enhancements include additional levels, richer animations, effects for interactions, persistent progress systems, and exporting the game for web deployment.

What I Would Do Differently:

In future projects I would lock in the jump mechanics, character controller and a mouse-camera in the very first session and run a targeted playtest before building any level content around it. I would also replace any hardcoded string paths or magic numbers in the global script from the outset, and ensure that the game boots successfully with zero manual configuration. Establishing an animation state machine early, rather than using sequential `if/elif` branches in `_physics_process`, would also make adding future animation states significantly easier.

A Literature and References

References

- [1] Bycer, J. (2020). *Game Design Deep Dive: Platformers* [Book]. CRC Press, Taylor & Francis Group. Covers the history of jumping mechanics, 3D platformer era, physics-driven design, and level structure.
- [2] Nintendo EAD. (1985). *Super Mario Bros.* [Video game]. Nintendo. Foundational reference for enemy interaction models (stomp mechanics), coin collection systems, and level progression.
- [3] Nintendo EAD. (1996). *Super Mario 64* [Video game]. Nintendo. Definitive reference for 3D platformer camera systems, analog movement, exploration-based level design, and combined-objective progression.
- [4] Naughty Dog. (1996). *Crash Bandicoot* [Video game]. Sony Computer Entertainment. Reference for linear 3D platforming, predictable enemy patterns, and moving platform timing puzzles.
- [5] Rare. (1998). *Banjo-Kazooie* [Video game]. Nintendo. Reference for collection-based progression systems and hub-world level structure.
- [6] Godot Foundation. “Godot Engine 4.x Official Documentation.” <https://docs.godotengine.org/>, 2025.
- [7] Godot Foundation. “Introduction to 3D.” <https://docs.godotengine.org/en/stable/tutorials/3d/>, 2025.
- [8] Godot Foundation. “Using CharacterBody3D.” https://docs.godotengine.org/en/stable/tutorials/3d/using_3d_characters/, 2025.
- [9] Godot Foundation. “AnimatableBody3D Class Reference.” https://docs.godotengine.org/en/stable/classes/class_animatablebody3d.html, 2025.
- [10] Godot Foundation. “Tween Class Reference.” https://docs.godotengine.org/en/stable/classes/class_tween.html, 2025.
- [11] Godot Foundation. “GDScript Reference.” <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>, 2025.
- [12] Masclef, M. (2023). “Dissecting the 3D Platformer Camera.” Game Developer Magazine. Analysis of third-person camera systems and implementation strategies.

B Resources and Assets

B.1 Audio Assets

Asset	Source	License
Jump SFX	Kenney.nl Audio Packs	CC0 Public Domain
Coin Collection SFX	Kenney.nl Audio Packs	CC0 Public Domain
Enemy Squash SFX	Pixabay Royalty-Free	Pixabay License
Hurt / Fall SFX	Pixabay Royalty-Free	Pixabay License
Button Click SFX	Kenney.nl Audio Packs	CC0 Public Domain
Game Start Jingle	Pixabay Royalty-Free	Pixabay License
Menu Background Music	Pixabay Royalty-Free	Pixabay License
Gameplay Background Music	Pixabay Royalty-Free	Pixabay License
Win / Game Over SFX	Kenney.nl Audio Packs	CC0 Public Domain

Table 2: Audio assets used in the project

All audio is managed through `sound_manager.gd`, which provides named functions for each sound event and handles music start/stop across scene transitions.

B.2 Visual Assets and Textures

Asset	Source	License
Robot Player (game-robot.glb)	BlendSwap – Infame-David, <i>Robot Goes to Jersey City</i>	Creative Commons
Coin Model (game-coin.glb)	BlenderKit – Game Gold Coin	Creative Commons
Enemy Models (Enemy.glb)	Quaternius Ultimate Platformer Pack	CC0 Public Domain
World Block Geometry (world_blocks.glb)	Custom Blender Models	Original Work
World Block Texture (Tiles138.*)	AmbientCG – Tiles138	CC0 Public Domain

Table 3: Visual assets used in the project

B.3 Code Libraries and Frameworks

Library	Version	License
Godot Engine	4.4.1	MIT
GDScript	Built-in	MIT

Table 4: Code libraries and dependencies

No external GDScript libraries or plugins are used. The project relies entirely on Godot’s built-in API.

B.4 Tools and Utilities

- **Godot Engine 4.4.1:** Primary development environment including 3D scene editor, script editor, animation editor, and integrated debugger
- **Blender:** Used for inspecting, adjusting, and re-exporting 3D models prior to import into Godot
- **Git / GitHub:** Version control and public project hosting at <https://github.com/SarthakBharad-Godot/3d-platformer>
- **AmbientCG Texture Downloader:** Used to obtain PBR texture sets for world block materials

B.5 Online Resources

- Godot Engine Official Documentation: <https://docs.godotengine.org/>
- Godot CharacterBody3D Tutorial: https://docs.godotengine.org/en/stable/tutorials/3d/using_3d_characters/
- GDScript Manual: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>
- Godot Community Forum: <https://forum.godotengine.org/>
- Quaternius Asset Library: <https://quaternius.com/>
- BlendSwap Community: <https://www.blendswap.com/>
- BlenderKit Asset Platform: <https://www.blenderkit.com/>
- AmbientCG Texture Library: <https://ambientcg.com/>
- Kenney.nl Audio Assets: <https://kenney.nl/assets/category:Audio>
- Pixabay Sound Effects: <https://pixabay.com/sound-effects/search/game/>

C Project Repository

GitHub Repository: <https://github.com/SarthakBharad-Godot/3d-platformer>

C.1 Repository Structure

```
res://
  scenes/
    character.tscn      # Player entity
    enemy.tscn          # Enemy entity with TopChecker and SideChecker
    coin.tscn           # Collectible coin with bounce animation
    platform.tscn       # Moving platform (AnimatableBody3D)
    level_one.tscn      # The playable level
    hud.tscn            # Heads-up display (coin + enemy counters)
    menu.tscn           # Main menu screen
    gameover.tscn       # Game over screen
    win.tscn            # Victory screen
    sound_manager.tscn  # Centralised audio node
  scripts/
    character.gd         # Player controller: movement, jump, camera, animations
    enemy.gd             # Enemy AI: patrol, edge detection, death, damage
    coin.gd              # Coin: rotation, pickup, win condition check
    platform.gd          # Moving platform: tween oscillation
    hud.gd               # HUD: initialises coin and enemy counter labels
    level_one.gd         # Level init: resets global state, starts audio
    menu.gd              # Menu: play button, stop music on transition
    gameover.gd          # Game Over: try again / back to menu buttons
    win.gd               # Win screen: try again / back to menu buttons
    sound_manager.gd     # Audio manager: all play/stop functions
    global.gd            # Autoload: coin/enemy counts, win condition, screenshot
  assets/
    models/
      game-robot.glb
      Enemy.glb
      game-coin.glb
      world_blocks.glb
    fonts/
    images/
    music/
    sound effects/
```

```
resources/
    environment.tres      # Sky and environment lighting settings
    world_blocks.tres     # PBR material for world geometry
project.godot            # Godot project configuration and input map
icon.svg                 # Project icon
README.md
```

C.2 Win Condition Constants

The win condition is defined by two constants in `global.gd`:

```
1 const COINS_TO_WIN = 67
2 const NO_OF_ENEMIES = 14
3
4 func check_win_condition() -> bool:
5     return coins >= COINS_TO_WIN and enemies >= NO_OF_ENEMIES
```

Listing 1: Win condition constants in `global.gd`

Both conditions must be satisfied simultaneously for the win screen to trigger.

C.3 How to Run

1. Install **Godot Engine 4.4.1** from <https://godotengine.org/download>
2. Clone the repository:

```
git clone https://github.com/SarthakBharad-Godot/3d-platformer
```
3. Open Godot, click **Import**, and select the `project.godot` file
4. Press **F5** or click the Play ► button to launch from the main menu

Game Controls:

Action	Input
Move Forward	W or ↑
Move Backward	S or ↓
Move Left	A or ←
Move Right	D or →
Jump	Spacebar or Enter
Rotate Camera Left	Numpad 1
Rotate Camera Right	Numpad 2
Take Screenshot	B